

Exercices corrigés

Networking Technologies and Applications, séances 1 à 3

BMEVITMAC09, automne 2026. Rédigé le 2026-09-28.

À retenir

- **Énoncés** : 39 questions en 12 niveaux, des conversions aux tables de routage, puis sérialisation et questions de cours.
- **Corrigés** : dans une partie séparée, avec les tableaux d'alignement binaire.

Statut. Ce ne sont pas des sujets officiels. Les seuls modèles d'exercices fournis par l'enseignant sont les quatre « Practical Examples » de `hta2p-ipv4-practical.pdf` (p. 11 à 14) et la conversion XML vers JSON ou YAML (`hta1p-protocols.pdf`, p. 26) ; les questions ci-dessous reprennent ces formats avec d'autres valeurs. Plusieurs utilisent des données réelles des slides : table route -n, en-tête capturé par Wireshark, fichier `kea.conf`. Les autres adresses sont des données d'exercice. Tous les résultats chiffrés ont été vérifiés avec le module Python `ipaddress` (script `exams/latex/calc_net.py`).

Méthode. Sur papier, sans calculatrice ni corrigé, en écrivant au moins l'octet intéressant en binaire. Le cours est dans « NET, cours et méthodes » ; les numéros de concept cités renvoient à ce document.

Tableaux binaires. **Bleu** : bits du préfixe ; **ocre** : partie hôte ; **|** : frontière.

Table des matières

1	Énoncés	3
1.1	Niveau 1. Conversions	3
1.2	Niveau 2. Classes et adresses spéciales	3
1.3	Niveau 3. Masques et CIDR	3
1.4	Niveau 4. Réseau, broadcast, plage d'hôtes	4
1.5	Niveau 5. Appartenance	4
1.6	Niveau 6. Découpage à taille fixe	4
1.7	Niveau 7. VLSM	5
1.8	Niveau 8. Agrégation	5
1.9	Niveau 9. Décision de routage	5
1.10	Niveau 10. En-tête et fragmentation	6
1.11	Niveau 11. Sérialisation	6
1.12	Niveau 12. Questions de cours	7
2	Corrigés : conversions, classes, masques	8
3	Corrigés : réseau, broadcast, appartenance	10
4	Corrigés : sous-réseaux et VLSM	13
5	Corrigés : agrégation et routage	15
6	Corrigés : en-tête, fragmentation, sérialisation	17
7	Corrigés : questions de cours	19

1. Énoncés

Niveau 1. Conversions

Q1. Convertir en binaire sur 8 bits : 45, 99 (par divisions successives), 250, 131.

Q2. Convertir en décimal : 10110110, 01111111, 11101010, 00010101.

Q3. Convertir 0x3F, 0xA7 et 0xFE en décimal et en binaire, puis 172 et 60 en hexadécimal.

Q4. Écrire en binaire (32 bits) et en hexadécimal les adresses 10.1.33.2 et 192.168.100.254 (deux adresses vues dans les captures de hta3p).

Q5. L'adresse 0xAC1F0A01 est écrite en hexadécimal. La donner en notation pointée. Est-elle privée ?

Niveau 2. Classes et adresses spéciales

Q6. Donner la classe historique et le type (privée, publique, loopback, multicast, réservée) de : 10.200.3.4 ; 172.32.1.1 ; 172.31.255.254 ; 192.168.0.1 ; 127.0.0.1 ; 224.0.0.5 ; 191.255.1.1 ; 223.1.1.1 ; 126.0.0.1 ; 240.0.0.1.

Q7. Justifier par le calcul les deux nombres de la slide pour la classe B : 16 384 réseaux et 65 534 hôtes par réseau.

Niveau 3. Masques et CIDR

Q8. Donner le masque de /13, /20, /27, /30, /9.

Q9. Donner la longueur de préfixe de 255.255.240.0, 255.128.0.0, 255.255.255.248, 255.254.0.0. Lesquels de ces masques sont invalides, et pourquoi : 255.255.0.255 ; 255.0.255.0 ; 255.255.255.160 ; 255.255.192.0 ?

Q10. Nombre d'hôtes d'un /19, d'un /23, d'un /28. Plus petit préfixe pour 300, 1000, 60, 62 et 63 hôtes.

Niveau 4. Réseau, broadcast, plage d'hôtes

Pour chaque adresse : masque, adresse réseau, broadcast, première et dernière adresse d'hôte, nombre d'hôtes.

Q11. 192.168.1.200/27

Q12. 172.16.45.14/20

Q13. 10.10.10.10/9

Q14. 131.7.255.254/23. Cette adresse peut-elle être attribuée à un hôte ?

Q15. 193.225.14.63/28. Cette adresse peut-elle être attribuée à un hôte ?

Q16. 100.64.17.130/26. Peut-on attribuer 100.64.17.191 à une autre machine du même réseau ?

Niveau 5. Appartenance

Q17. Lesquelles de ces adresses sont dans le même sous-réseau que 10.20.30.40/21 : (a) 10.20.31.200 ; (b) 10.20.24.1 ; (c) 10.20.39.1 ; (d) 10.21.30.40 ; (e) 10.20.32.0 ?

Q18. Deux machines ont les adresses 192.168.5.130 et 192.168.5.100, masque /25. Peuvent-elles échanger directement, sans routeur ? Et avec un masque /24 ?

Niveau 6. Découpage à taille fixe

Q19. Découper 192.168.30.0/24 en 4 sous-réseaux de même taille. Pour chacun : réseau, plage d'hôtes, broadcast.

Q20. Découper 10.0.0.0/16 en sous-réseaux d'au moins 1000 hôtes chacun, le plus nombreux possible. Préfixe, nombre de sous-réseaux, 5^e sous-réseau (plage complète) et dernier sous-réseau.

Q21. Une entreprise veut 12 sous-réseaux égaux dans 172.18.0.0/16, les plus grands possible. Préfixe, nombre d'hôtes par sous-réseau, sous-réseau numéro 10 en comptant à partir de 0.

Niveau 7. VLSM

Q22. Plan d'adressage dans 172.16.8.0/22 pour : LAN A 400 hôtes, LAN B 120, LAN C 60, LAN D 30, deux liens point à point entre routeurs. Donner réseau, masque, plage, broadcast, et ce qui reste libre.

Q23. Plan d'adressage dans 192.168.100.0/24 pour : Production 60 hôtes, Bureaux 60, Wi-Fi 28, Serveurs 12, Administration 5.

Q24. Peut-on loger dans 200.10.10.0/24 trois réseaux de 120, 70 et 50 hôtes ? Justifier.

Niveau 8. Agrégation

Q25. Agréger 172.16.12.0/24, 172.16.13.0/24, 172.16.14.0/24 et 172.16.15.0/24, tous vers la même interface.

Q26. Table de routage : (1) 200.20.0.0/17 eth1 ; (2) 200.20.128.0/18 eth1 ; (3) 200.20.192.0/18 eth1 ; (4) 200.21.0.0/16 eth2. Compresser la table au maximum. Que deviendrait-elle si (4) sortait aussi par eth1 ?

Q27. (A) 192.168.2.0/24 et 192.168.3.0/24 vers la même interface : peut-on les fusionner ? (B) Même question pour 192.168.3.0/24 et 192.168.4.0/24. Quel est, dans le cas B, le plus petit préfixe qui couvre les deux, et pourquoi n'est-ce pas une bonne agrégation ?

Niveau 9. Décision de routage

Q28. Table : 0.0.0.0/0 vers R0 ; 152.66.0.0/16 eth1 ; 152.66.128.0/17 eth2 ; 152.66.160.0/19 eth3 ; 152.66.163.0/24 eth4. Interface de sortie pour : 152.66.163.9 ; 152.66.170.1 ; 152.66.200.1 ; 152.66.5.5 ; 152.67.0.1.

Q29. Voici la table route -n réelle de hta3p-ipv4-tools.pdf, p. 12 (colonnes Metric, Ref, Use omises) :

Destination	Gateway	Genmask	Flags	Iface
0.0.0.0	192.168.100.254	0.0.0.0	UG	enp8s0f0
1.1.1.1	192.168.100.254	255.255.255.255	UGH	enp8s0f0
8.8.8.8	192.168.100.254	255.255.255.255	UGH	enp8s0f0
10.1.11.0	0.0.0.0	255.255.255.0	U	enp6s0

10.1.33.0	0.0.0.0	255.255.255.0	U	enp3s0
172.17.0.0	0.0.0.0	255.255.0.0	U	docker0
192.168.100.0	0.0.0.0	255.255.255.0	U	enp8s0f0
192.168.100.254	0.0.0.0	255.255.255.255	UH	enp8s0f0

Pour chaque destination, dire l'interface et si le paquet passe par une passerelle : 8.8.8.8; 10.1.33.2; 172.17.5.5; 9.9.9.9; 192.168.100.77; 10.1.12.1.

Niveau 10. En-tête et fragmentation

Q30. Voici les 20 premiers octets IP de la capture Wireshark de hta3p, p. 22 :

```
45 00 00 3c 6f 89 00 00 02 11 30 8d c0 a8 64 0f ac 43 47 a0
```

Donner version, longueur d'en-tête, longueur totale, Identification, drapeaux et offset, TTL, protocole, adresses source et destination. Question bonus (méthode hors slides) : vérifier la somme de contrôle.

Q31. Un paquet IPv4 de 3000 octets (en-tête de 20 octets, sans options) doit traverser un lien de MTU 1000. Donner pour chaque fragment : octets de données, Total Length, Fragment Offset, MF. Que se passe-t-il si DF vaut 1 ?

Niveau 11. Sérialisation

Q32. Convertir ce document XML (données d'exercice) en JSON puis en YAML.

```
<host>
  <name>nb</name>
  <interface>ethBA</interface>
  <lease>
    <address>10.0.1.10</address>
    <prefix>24</prefix>
    <router>10.0.1.1</router>
    <valid>20</valid>
  </lease>
  <dns>
    <server>10.0.1.1</server>
    <server>10.0.1.2</server>
  </dns>
</host>
```

Q33. Même exercice avec la table de l'exemple 3 des slides :

```
<routingTable>
  <route><dest>170.80.0.0/13</dest><iface>eth7</iface></route>
  <route><dest>170.88.0.0/13</dest><iface>eth2</iface></route>
  <route><dest>170.84.0.0/14</dest><iface>eth11</iface></route>
</routingTable>
```

Niveau 12. Questions de cours

Q34. ICMP. (a) Numéro de protocole dans l'en-tête IP. (b) Types de l'Echo Request et de l'Echo Reply. (c) Quel message les routeurs intermédiaires renvoient-ils à traceroute, et pourquoi? (d) Quel message reçoit la source si DF vaut 1 et que la MTU est trop petite? (e) Deux problèmes de sécurité d'ICMP.

Q35. DHCP. (a) Nommer les quatre messages de l'échange de base, dans l'ordre, et dire lesquels sont diffusés en broadcast. (b) Transport et ports. (c) Pourquoi le Request est-il en broadcast? (d) Que signifie « état mou »? (e) Dans la démo Kea, combien d'adresses le pool peut-il distribuer, et quelle passerelle est annoncée?

Q36. DNS. (a) Types d'enregistrement pour une adresse IPv4, IPv6, un serveur de courrier. (b) Transports utilisés. (c) Comment une requête atteint-elle le serveur autoritaire? (d) Trois outils de requête.

Q37. Outils. (a) Commande ip pour ajouter 10.0.20.30/24 à eth0. (b) Lister les routes en numérique. (c) Lister les connexions ouvertes avec les processus. (d) Capturer le trafic en console. (e) Lire le bilan de ping de la slide p. 18 : 24 packets transmitted, 24 received, 0% packet loss et rtt min/avg/max/mdev = 1.062/1.282/1.443/0.097 ms.

Q38. Modèles en couches. (a) Nom des PDU aux couches transport, réseau, liaison et physique d'OSI. (b) À quelles couches OSI correspond la couche Application de TCP/IP? (c) Dans quel ordre les en-têtes apparaissent-ils sur le fil pour une requête HTTP sur Ethernet?

Q39. QoS. Un client annonce un débit moyen de 4 Mbit/s, un débit maximal de 10 Mbit/s et des rafales de 2 s. Dimensionner un token bucket selon la règle de la slide. Quelle différence de comportement avec un leaky bucket de mêmes paramètres?

2. Corrigés : conversions, classes, masques

Q1. $45 : 32 + 8 + 4 + 1 : 00101101$. $250 : 128 + 64 + 32 + 16 + 8 + 2 : 11111010$.
 $131 : 128 + 2 + 1 : 10000011$.

99 par divisions : $99 = 2 \times 49 + 1$; $49 = 2 \times 24 + 1$; $24 = 2 \times 12 + 0$; $12 = 2 \times 6 + 0$;
 $6 = 2 \times 3 + 0$; $3 = 2 \times 1 + 1$; $1 = 2 \times 0 + 1$. Restes de bas en haut : 1100011,
soit 01100011 sur 8 bits ($64 + 32 + 2 + 1 = 99$).

Q2. $10110110 = 128 + 32 + 16 + 4 + 2 = 182$; $01111111 = 127$; 11101010
 $= 128 + 64 + 32 + 8 + 2 = 234$; $00010101 = 16 + 4 + 1 = 21$.

Q3. $0x3F = 3 \times 16 + 15 = 63$, 0011 1111. $0xA7 = 10 \times 16 + 7 = 167$, 1010
0111. $0xFE = 15 \times 16 + 14 = 254$, 1111 1110. $172 = 10 \times 16 + 12 : 0xAC$.
 $60 = 3 \times 16 + 12 : 0x3C$.

Q4.

Adresse	Binaire	Hex
10.1.33.2	00001010.00000001.00100001.00000010	0x0A012102
192.168.100.254	11000000.10101000.01100100.11111110	0xC0A864FE

$33 = 32 + 1$; $100 = 64 + 32 + 4 = 0x64$; $254 = 255 - 1 = 0xFE$.

Q5. AC, 1E, 0A, 01 : $10 \times 16 + 12 = 172$, $16 + 15 = 31$, 10, 1. Adresse
172.31.10.1. Privée : 172.16.0.0/12 couvre 172.16 à 172.31, et 31 =
00011111 commence bien par 0001.

Q6.

Adresse	1 ^{er} octet	Classe, type
10.200.3.4	00001010	A, privée (10.0.0.0/8)
172.32.1.1	10101100	B, publique (32 est hors de 16 à 31)
172.31.255.254	10101100	B, privée
192.168.0.1	11000000	C, privée
127.0.0.1	01111111	loopback (réseau 127 réservé)
224.0.0.5	11100000	D, multicast
191.255.1.1	10111111	B, publique
223.1.1.1	11011111	C, publique
126.0.0.1	01111110	A, publique
240.0.0.1	11110000	E, réservée

Q7. Classe B : préfixe de 16 bits dont les 2 premiers sont fixés à 10, il reste 14 bits libres : $2^{14} = 16\,384$ réseaux. Partie hôte de 16 bits : $2^{16} = 65\,536$ combinaisons, moins l'adresse de réseau (tout à 0) et le broadcast (tout à 1) : 65 534.

Q8. /13 = 8 + 5 : 255.248.0.0. /20 = 16 + 4 : 255.255.240.0. /27 = 24 + 3 : 255.255.255.224. /30 = 24 + 6 : 255.255.255.252. /9 = 8 + 1 : 255.128.0.0.

Q9. 255.255.240.0 : 16 + 4 = /20. 255.128.0.0 : 8 + 1 = /9. 255.255.255.248 : 24 + 5 = /29. 255.254.0.0 : 8 + 7 = /15.

Invalides : 255.255.0.255 (00000000 suivi de 11111111); 255.0.255.0 (même défaut); 255.255.255.160 (160 = 10100000, un 0 entre deux 1). 255.255.192.0 est valide : /18.

Q10. /19 : $2^{13} - 2 = 8190$. /23 : $2^9 - 2 = 510$. /28 : $2^4 - 2 = 14$.

300 hôtes : $2^9 - 2 = 510 \geq 300$ et $2^8 - 2 = 254 < 300$: /23. 1000 : $2^{10} - 2 = 1022$: /22. 60 : $2^6 - 2 = 62$: /26. 62 : /26 aussi, tout juste. 63 : $62 < 63$, il faut $2^7 - 2 = 126$: /25.

3. Corrigés : réseau, broadcast, appartenance

Q11.

Adresse	192.168.1.200	11000000.10101000.00000001.110 01000
Masque /27	255.255.255.224	11111111.11111111.11111111.111 00000
Réseau (ET)	192.168.1.192	11000000.10101000.00000001.110 00000
Broadcast	192.168.1.223	11000000.10101000.00000001.110 11111

Masque 255.255.255.224, $m = 32$; $192 \leq 200 < 224$. Réseau **192.168.1.192**, broadcast **192.168.1.223**, hôtes 192.168.1.193 à 192.168.1.222, soit 30.

Q12.

Adresse	172.16.45.14	10101100.00010000.0010 1101.00001110
Masque /20	255.255.240.0	11111111.11111111.1111 0000.00000000
Réseau (ET)	172.16.32.0	10101100.00010000.0010 0000.00000000
Broadcast	172.16.47.255	10101100.00010000.0010 1111.11111111

Masque 255.255.240.0, octet intéressant le 3^e, $m = 16$; $32 \leq 45 < 48$; en binaire $45 = 00101101$, ET $11110000 = 00100000 = 32$. Réseau **172.16.32.0**, broadcast **172.16.47.255**, hôtes 172.16.32.1 à 172.16.47.254, soit $2^{12} - 2 = 4094$.

Q13.

Adresse	10.10.10.10	00001010.0 0001010.00001010.00001010
Masque /9	255.128.0.0	11111111.1 0000000.00000000.00000000
Réseau (ET)	10.0.0.0	00001010.0 0000000.00000000.00000000
Broadcast	10.127.255.255	00001010.0 11111111.11111111.11111111

Masque 255.128.0.0, 2^e octet, $m = 128$; $0 \leq 10 < 128$. Réseau **10.0.0.0**, broadcast **10.127.255.255**, hôtes 10.0.0.1 à 10.127.255.254, soit $2^{23} - 2 = 8\,388\,606$.

Q14.

Adresse	131.7.255.254	10000011.00000111.11111111 1.11111110
Masque /23	255.255.254.0	11111111.11111111.11111111 0.00000000
Réseau (ET)	131.7.254.0	10000011.00000111.11111111 0.00000000
Broadcast	131.7.255.255	10000011.00000111.11111111 1.11111111

Masque 255.255.254.0, 3^e octet, $m = 2$; $254 \leq 255 < 256$. Réseau **131.7.254.0**, broadcast **131.7.255.255**, hôtes 131.7.254.1 à 131.7.255.254, soit 510. L'adresse 131.7.255.254 est la **dernière adresse d'hôte** : attribuable. Un 4^e octet à 255 ou 0 n'est pas forcément interdit dans un réseau plus grand qu'un /24 : 131.7.254.255 est ici une adresse d'hôte ordinaire.

Q15.

Adresse	193.225.14.63	11000001.11100001.00001110.0011 1111
Masque /28	255.255.255.240	11111111.11111111.11111111.1111 0000
Réseau (ET)	193.225.14.48	11000001.11100001.00001110.0011 0000
Broadcast	193.225.14.63	11000001.11100001.00001110.0011 1111

Masque 255.255.255.240, $m = 16$; $48 \leq 63 < 64$. Réseau 193.225.14.48, broadcast **193.225.14.63** : c'est l'adresse donnée. **Non attribuable**. Hôtes .49 à .62, soit 14.

Q16.

Adresse	100.64.17.130	01100100.01000000.00010001.10 000010
Masque /26	255.255.255.192	11111111.11111111.11111111.11 000000
Réseau (ET)	100.64.17.128	01100100.01000000.00010001.10 000000
Broadcast	100.64.17.191	01100100.01000000.00010001.10 111111

Masque 255.255.255.192, $m = 64$; $128 \leq 130 < 192$. Réseau 100.64.17.128, broadcast 100.64.17.191, hôtes .129 à .190, soit 62. 100.64.17.191 est le broadcast : **non**.

Q17.

Réf.	10.20.30.40	00001010.00010100.00011 110.00101000
(a)	10.20.31.200	00001010.00010100.00011 111.11001000
(b)	10.20.24.1	00001010.00010100.00011 000.00000001
(c)	10.20.39.1	00001010.00010100.00100 111.00000001
(d)	10.21.30.40	00001010.00010101.00011 110.00101000
(e)	10.20.32.0	00001010.00010100.00100 000.00000000

Réseau de référence : $m = 8$ au 3^e octet, $24 \leq 30 < 32$: 10.20.24.0/21, plage 10.20.24.0 à 10.20.31.255 (5 premiers bits du 3^e octet : 00011). (a) 31 = 00011111 : **oui**. (b) 24 = 00011000 : **oui**. (c) 39 = 00100111 : **non**, réseau 10.20.32.0/21. (d) 2^e octet 21 : **non**. (e) 32 = 00100000 : **non**, c'est même l'adresse du réseau voisin 10.20.32.0/21.

Q18.

Hôte 1	192.168.5.130	11000000.10101000.00000101.1 0000010
Hôte 2	192.168.5.100	11000000.10101000.00000101.0 1100100

En /25, le 25^e bit diffère : 1 pour .130, 0 pour .100. Les réseaux sont 192.168.5.128/25 et 192.168.5.0/25. **Pas d'échange direct**, il faut passer par un routeur. En /24, les 24 premiers bits sont identiques : même réseau 192.168.5.0/24, échange direct.

4. Corrigés : sous-réseaux et VLSM

Q19. 4 sous-réseaux : $s = 2$ bits, préfixe /26, masque 255.255.255.192, 62 hôtes chacun.

Bits empruntés	Réseau	Hôtes	Broadcast
00	192.168.30.0/26	.1 à .62	.63
01	192.168.30.64/26	.65 à .126	.127
10	192.168.30.128/26	.129 à .190	.191
11	192.168.30.192/26	.193 à .254	.255

Q20. 1000 hôtes : $2^{10} - 2 = 1022$, donc $h = 10$, préfixe /22 (255.255.252.0). Bits empruntés : $22 - 16 = 6$, soit $2^6 = 64$ sous-réseaux, pas de 4 au 3^e octet. 5^e sous-réseau ($k = 4$) : $4 \times 4 = 16$: **10.0.16.0/22**, hôtes 10.0.16.1 à 10.0.19.254, broadcast 10.0.19.255. Dernier ($k = 63$) : $63 \times 4 = 252$: **10.0.252.0/22**, broadcast 10.0.255.255.

Q21. $2^3 = 8 < 12 \leq 16 = 2^4$: $s = 4$, préfixe /20 (255.255.240.0), 16 sous-réseaux (4 de réserve), $2^{12} - 2 = 4094$ hôtes chacun, pas de 16 au 3^e octet. $k = 10$: $10 \times 16 = 160$: **172.18.160.0/20**, hôtes 172.18.160.1 à 172.18.175.254, broadcast 172.18.175.255.

Q22. Tailles : $400 \rightarrow 512$ (/23), $120 \rightarrow 128$ (/25), $60 \rightarrow 64$ (/26), $30 \rightarrow 32$ (/27), liens $\rightarrow 4$ (/30). Total $512 + 128 + 64 + 32 + 4 + 4 = 744 \leq 1024$.

Réseau	Préfixe	Masque	Plage d'hôtes	Broadcast
LAN A	172.16.8.0/23	255.255.254.0	8.1 à 9.254	9.255
LAN B	172.16.10.0/25	255.255.255.128	10.1 à 10.126	10.127
LAN C	172.16.10.128/26	255.255.255.192	10.129 à 10.190	10.191
LAN D	172.16.10.192/27	255.255.255.224	10.193 à 10.222	10.223
Lien 1	172.16.10.224/30	255.255.255.252	10.225, 10.226	10.227
Lien 2	172.16.10.228/30	255.255.255.252	10.229, 10.230	10.231

(Plages abrégées : « 8.1 » signifie 172.16.8.1.) Libre : de 172.16.10.232 à 172.16.11.255.

Q23. Tailles : $60 \rightarrow 64$ (/26, 62 hôtes) deux fois, $28 \rightarrow 32$ (/27), $12 \rightarrow 16$ (/28), $5 \rightarrow 8$ (/29, 6 hôtes). Total $64 + 64 + 32 + 16 + 8 = 184 \leq 256$.

Réseau	Préfixe	Hôtes	Broadcast
Production	192.168.100.0/26	.1 à .62	.63
Bureaux	192.168.100.64/26	.65 à .126	.127
Wi-Fi	192.168.100.128/27	.129 à .158	.159
Serveurs	192.168.100.160/28	.161 à .174	.175
Administration	192.168.100.176/29	.177 à .182	.183

Libre : 192.168.100.184 à 192.168.100.255.

Q24. 120 demande un /25 (126 hôtes, 128 adresses). 70 ne tient pas dans un /26 (62 hôtes) : il faut aussi un /25. Deux /25 occupent déjà les 256 adresses du /24 ; il ne reste rien pour les 50 hôtes (un /26 de 64 adresses). Total nécessaire $128 + 128 + 64 = 320 > 256$: **impossible**.

5. Corrigés : agrégation et routage

Q25.

/24	172.16.12.0	10101100.00010000.000011 00.00000000
/24	172.16.13.0	10101100.00010000.000011 01.00000000
/24	172.16.14.0	10101100.00010000.000011 10.00000000
/24	172.16.15.0	10101100.00010000.000011 11.00000000
Résumé /22	172.16.12.0	10101100.00010000.000011 00.00000000

12 = 00001100, 15 = 00001111 : les 6 premiers bits du 3^e octet sont communs, les 2 derniers prennent les 4 valeurs. Résumé exact : **172.16.12.0/22**. (12 est un multiple de 4 : le bloc est aligné.)

Q26.

- (1) /17 200.20.0.0 11001000.00010100.0|0000000.00000000
- (2) /18 200.20.128.0 11001000.00010100.10|000000.00000000
- (3) /18 200.20.192.0 11001000.00010100.11|000000.00000000
- (4) /16 200.21.0.0 11001000.00010101.|00000000.00000000

(2) et (3) : /18, 3^e octets 10000000 et 11000000, seul le 18^e bit diffère, même interface : fusion en 200.20.128.0/17. Avec (1) 200.20.0.0/17 : seul le 17^e bit diffère : fusion en **200.20.0.0/16** vers eth1. (4) 200.21.0.0/16 sort par eth2 : pas de fusion. Table finale : 200.20.0.0/16 eth1 ; 200.21.0.0/16 eth2.

Si (4) sortait par eth1 : 20 = 00010100 et 21 = 00010101 ne diffèrent que par le 16^e bit : une seule ligne, **200.20.0.0/15** eth1.

Q27.

- A 192.168.2.0 11000000.10101000.0000001|0.00000000
- B 192.168.3.0 11000000.10101000.0000001|1.00000000
- C 192.168.3.0 11000000.10101000.00000|011.00000000
- D 192.168.4.0 11000000.10101000.00000|100.00000000

(A) 2 = 00000010, 3 = 00000011 : seul le 24^e bit diffère, fusion en **192.168.2.0/23**. (B) 3 = 00000011, 4 = 00000100 : ils diffèrent dès le 22^e bit. Contigus mais non alignés : pas de /23. Le plus petit préfixe commun est **192.168.0.0/21** (192.168.0.0 à 192.168.7.255), qui couvre aussi 192.168.0, 1, 2, 5, 6 et 7 : on enverrait vers cette interface du trafic qui ne

lui est pas destiné. Il faut garder les deux lignes.

Q28.

Destination	Lignes qui correspondent	Sortie
152.66.163.9	/0, /16, /17, /19, /24	eth4 (/24)
152.66.170.1	/0, /16, /17, /19	eth3 (/19)
152.66.200.1	/0, /16, /17	eth2 (/17)
152.66.5.5	/0, /16	eth1 (/16)
152.67.0.1	/0	R0 (défaut)

Repères : /17 couvre 152.66.128.0 à 152.66.255.255; /19 couvre 152.66.160.0 à 152.66.191.255 ($m = 32$ au 3^e octet); /24 couvre 152.66.163.x. 170 est entre 160 et 191 mais 200 non.

Q29.

Destination	Ligne retenue	Résultat
8.8.8.8	8.8.8.8/32, UGH	enp8s0f0, via 192.168.100.254
10.1.33.2	10.1.33.0/24, U	enp3s0, directe
172.17.5.5	172.17.0.0/16, U	docker0, directe
9.9.9.9	0.0.0.0/0, UG	enp8s0f0, via 192.168.100.254
192.168.100.77	192.168.100.0/24, U	enp8s0f0, directe
10.1.12.1	0.0.0.0/0, UG	enp8s0f0, via 192.168.100.254

Pour 8.8.8.8, la route par défaut correspond aussi, mais le /32 est plus long. 10.1.12.1 n'est ni dans 10.1.11.0/24 ni dans 10.1.33.0/24 : seule la route par défaut reste.

6. Corrigés : en-tête, fragmentation, sérialisation

Q30.

Octets	Champ	Valeur
45	version, IHL	4 ; $5 \times 4 = 20$ octets
00	DSCP, ECN	0, 0
00 3c	Total Length	60 octets (40 de données UDP)
6f 89	Identification	$6 \times 4096 + 15 \times 256 + 8 \times 16 + 9 = 28\,553$
00 00	Flags, Offset	DF = 0, MF = 0, offset 0
02	TTL	2
11	Protocol	17, UDP
30 8d	Checksum	0x308D
c0 a8 64 0f	Source	192.168.100.15
ac 43 47 a0	Destination	172.67.71.160

Ce sont les valeurs affichées par Wireshark sur la slide.

Bonus (RFC 1071, non traité dans les slides) : additionner les dix mots de 16 bits : $0x4500 + 0x003C + 0x6F89 + 0x0000 + 0x0211 + 0x308D + 0xC0A8 + 0x640F + 0xAC43 + 0x47A0 = 0x2FFFD$. Replier la retenue : $0xFFFFD + 0x2 = 0xFFFF$. Le résultat tout à 1 signifie que l'en-tête est intact. (Sans le champ de contrôle, la somme repliée vaut $0xCF72$, dont le complément à 1 est bien $0x308D$.)

Q31. Données : $3000 - 20 = 2980$. Par fragment : $1000 - 20 = 980$, ramené au multiple de 8 inférieur : $\lfloor 980/8 \rfloor \times 8 = 122 \times 8 = 976$.

Fragment	Données	Total Length	Offset	MF
1	976	996	0	1
2	976	996	122	1
3	976	996	244	1
4	52	72	366	0

Contrôle : $3 \times 976 + 52 = 2980$; $366 \times 8 = 2928$ octets déjà envoyés. Tous gardent la même Identification. Si DF = 1 : le routeur jette le paquet et

peut renvoyer un ICMP Destination Unreachable, sous-type « fragmentation needed » (type 3).

Q32. Racine host omise, comme sur la slide ; server répété devient une liste ; prefix et valid sont des nombres.

```
{
  "name": "nb",
  "interface": "ethBA",
  "lease": {
    "address": "10.0.1.10",
    "prefix": 24,
    "router": "10.0.1.1",
    "valid": 20
  },
  "dns": {
    "server": [ "10.0.1.1",
                "10.0.1.2" ]
  }
}
```

```
name: nb
interface: ethBA
lease:
  address: 10.0.1.10
  prefix: 24
  router: 10.0.1.1
  valid: 20
dns:
  server:
    - 10.0.1.1
    - 10.0.1.2
```

Variante acceptable : "dns": ["10.0.1.1", "10.0.1.2"], en supprimant la balise répétée comme la slide le fait pour SlideTitle. Annoncer le choix.

Q33. Chaque route est un élément composé : la liste contient des objets.

```
{
  "route": [
    { "dest": "170.80.0.0/13",
      "iface": "eth7" },
    { "dest": "170.88.0.0/13",
      "iface": "eth2" },
    { "dest": "170.84.0.0/14",
      "iface": "eth11" }
  ]
}
```

```
route:
  - dest: 170.80.0.0/13
    iface: eth7
  - dest: 170.88.0.0/13
    iface: eth2
  - dest: 170.84.0.0/14
    iface: eth11
```

En YAML, le tiret ouvre un élément de liste et les clés suivantes, alignées sous la première, appartiennent au même objet.

7. Corrigés : questions de cours

Source : Q34 : hta3 p. 4 à 7 ; Q35 : hta3 p. 8 à 10 et kea.conf ; Q36 : hta3 p. 11 à 18 ; Q37 : hta3p p. 9 à 18 ; Q38 : hta1p p. 3 à 6 ; Q39 : hta1p p. 18 et 19

Q34. (a) Protocol = 1. (b) Echo Request type 8, Echo Reply type 0. (c) Time Exceeded (type 11), car le TTL, envoyé à 1, 2, 3..., tombe à 0 sur le routeur de rang correspondant ; chaque réponse révèle un saut. (d) Destination Unreachable (type 3), sous-type « fragmentation needed ». (e) Au choix : pas d'authentification (faux Redirect), ping flood, tunnel ping, cartographie.

Q35. (a) Discover (broadcast liaison et IP), Offer, Request (broadcast), Ack. (b) UDP, serveur 67, client 68. (c) Pour prévenir tous les serveurs de l'offre retenue. (d) Le bail expire s'il n'est pas renouvelé ; utiliser une adresse expirée pose problème si le serveur l'a redonnée. (e) Pool 10.0.1.10 à 10.0.1.30 : $30 - 10 + 1 = 21$ adresses ; passerelle (routers) 10.0.1.1.

Q36. (a) A, AAAA, MX. (b) UDP/53 d'origine, TCP/53 (messages longs, fiabilité), TLS, HTTPS. (c) Le client interroge son résolveur local (souvent appris par DHCP) ; la requête monte dans l'arbre puis redescend jusqu'au serveur autoritaire, ou s'arrête à une réponse en cache. (d) host, nslookup, dig.

Q37. (a) `ip addr add 10.0.20.30/24 dev eth0`. (b) `route -n` (ou `netstat -r`). (c) `netstat -pnaut`. (d) `tcpdump` (ou `tshark`). (e) 24 requêtes, 24 réponses, aucune perte ; aller-retour minimal 1,062 ms, moyen 1,282 ms, maximal 1,443 ms, écart moyen 0,097 ms.

Q38. (a) Segment, Packet, Frame, Bit. (b) Application, Présentation et une partie de Session (selon le schéma de la slide p. 6). (c) Ethernet, IP, TCP, puis HTTP : la couche basse encapsule en dernier.

Q39. $K = 4$ Mbit/s (débit moyen) ; $B = (10 - 4) \times 2 = 12$ Mbit. Le token bucket laisse passer les rafales tant qu'il reste des jetons et rejette au-delà (mesure, passe ou échoue). Le leaky bucket met en forme : la sortie ne dépasse jamais K , l'excès attend dans une file et n'est rejeté que seau plein.